



**HỌC VIỆN
NGÂN HÀNG**
BANKING ACADEMY OF VIETNAM

Web Design

Lecture 10 - JavaScript



6.5. Loops and conditional statements

6.6. Functions

6.7. JavaScript Objects



JavaScript For Loop

JavaScript For In

JavaScript While Loop

JavaScript if, else, and else if

JavaScript Break and Continue



Loops can execute a block of code a number of times.

Loops are handy, if you want to run the same code over and over again, each time with a different value.

Instead of writing:

```
text += cars[0] + "<br>";  
text += cars[1] + "<br>";  
text += cars[2] + "<br>";  
text += cars[3] + "<br>";  
text += cars[4] + "<br>";  
text += cars[5] + "<br>";
```

You can write:

```
for (let i = 0; i < cars.length; i++) {  
  text += cars[i] + "<br>";  
}
```



The general statement:

```
for (expression 1; expression 2; expression 3) {  
    // code block to be executed  
}
```

- Expression 1 is executed (one time) before the execution of the code block.
- Expression 2 defines the condition for executing the code block.
- Expression 3 is executed (every time) after the code block has been executed.

```
for (let i = 0; i < 5; i++) {  
    text += "The number is " + i + "<br>";  
}
```



Create an array named ***studentList***, with 5 elements given by your 5 friends' name

Write a **for loop** to alert your friends that tomorrow is the deadline for submitting the project assignment

*“Hey ” + **studentList[i]** + “, today is ” + **today_value** + “, deadline is coming!”*



Conditional statements are used to perform **different actions** based on **different conditions**.

In JavaScript we have the following conditional statements:

- Use **if** to specify a block of code to be executed, if a specified condition is true
- Use **else** to specify a block of code to be executed, if the same condition is false
- Use **else if** to specify a new condition to test, if the first condition is false
- Use **switch** to specify many alternative blocks of code to be executed



Use the if statement to specify a block of JavaScript code to be executed if a condition is true.

```
if (condition) {  
    // block of code to be executed if the condition is true  
}
```

For example.

```
if (hour < 18) {  
    greeting = "Good day";  
}
```



Use the else statement to specify a block of code to be executed if the condition is false.

```
if (condition) {  
    // block of code to be executed if the condition is true  
} else {  
    // block of code to be executed if the condition is false  
}
```

E.g:

```
if (hour < 18) {  
    greeting = "Good day";  
} else {  
    greeting = "Good evening";  
}
```



Use the else if statement to specify a new condition if the first condition is false.

```
if (condition1) {  
    // block of code to be executed if condition1 is true  
} else if (condition2) {  
    // block of code to be executed if the condition1 is false and condition2 is true  
} else {  
    // block of code to be executed if the condition1 is false and condition2 is false  
}
```

E.g:

```
if (time < 10) {  
    greeting = "Good morning";  
} else if (time < 20) {  
    greeting = "Good day";  
} else {  
    greeting = "Good evening";  
}
```



Exercise in page 606



JavaScript Functions

Function Invocation

Function Return



A JavaScript function is a block of code designed to perform a particular task.

A JavaScript function is executed when "something" invokes it (calls it).

```
function name(parameter1, parameter2, parameter3) {  
    // code to be executed  
}
```

E.g.:

```
// Function to compute the product of p1 and p2  
function myFunction(p1, p2) {  
    return p1 * p2;  
}
```



The code inside the function will execute when "something" invokes (calls) the function:

- When an event occurs (when a user clicks a button)
- When it is invoked (called) from JavaScript code
- Automatically (self invoked)



When JavaScript reaches a return statement, the function will stop executing.

If the function was invoked from a statement, JavaScript will "return" to execute the code after the invoking statement.

Functions often compute a return value. The return value is "returned" back to the "caller":

```
// Function is called, the return value will end up in x
let x = myFunction(4, 3);

function myFunction(a, b) {
  // Function returns the product of a and b
  return a * b;
}
```



Hundreds of predefined functions are built into JavaScript, including these:

alert(), confirm(), and prompt(): These functions trigger browser-level dialog boxes.

Date(): Returns the current date and time.

parseInt("123"): This function will, among other things, take a string data type containing numbers and turn it into a number data type. The string is passed to the function as an argument.

... and many other built-in functions



To create a custom function, we type the function keyword followed by a name for the function, followed by opening and closing parentheses, followed by opening and closing curly brackets:

```
function name() {  
    // Our function code goes here.  
}
```

Just as with variables and arrays, the function's name can be anything you want, but all the same naming syntax rules apply. If we were to create a function that just alerts some text (which is a little redundant, I know), it would look like this:

```
function foo() {  
    alert("Our function just ran!");  
    // This code won't run until we call the function 'foo()'  
}
```



Viết file **[Ho_ten_sv].html** có các thành phần tối thiểu và mã JavaScript thực hiện các yêu cầu sau:

Yêu cầu người dùng nhập vào một mật khẩu bằng prompt(). Viết hàm kiểm tra mật khẩu theo các tiêu chí sau:

- Ít nhất 8 ký tự.
- Chứa ít nhất một chữ cái viết hoa (A-Z).
- Chứa ít nhất một chữ cái viết thường (a-z).
- Chứa ít nhất một chữ số (0-9).
- Chứa ít nhất một ký tự đặc biệt (ví dụ: @, #, \$, % ...).

Nếu mật khẩu không hợp lệ, hiển thị thông báo lỗi tương ứng và yêu cầu nhập lại.

Nếu mật khẩu hợp lệ, hiển thị thông báo "Mật khẩu hợp lệ!" kèm theo độ mạnh của mật khẩu:

- Yếu: Chỉ vừa đủ 8 ký tự
- Trung bình: Đáp ứng tất cả điều kiện nhưng dưới 12 ký tự.
- Mạnh: Đáp ứng tất cả điều kiện và có từ 12 ký tự trở lên.

Thử nghiệm với các tình huống khác nhau trước khi nộp lại file **[Ho_ten_sv].html**.



Real Life Objects, Properties, and Methods

Object Definition

Object Properties

Object Methods

The **this** keyword



In real life, a car is an **object**.

A car has **properties** like ***weight*** and ***color***, and **methods** like ***start*** and ***stop***:

Object	Properties	Methods
	car.name = Fiat car.model = 500 car.weight = 850kg car.color = white	car.start() car.drive() car.brake() car.stop()



All cars have the **same properties**, but the **property values differ** from car to car.

All cars have the **same methods**, but the **methods are performed at different times**.

Object	Properties	Methods
	<code>car.name = Fiat</code> <code>car.model = 500</code> <code>car.weight = 850kg</code> <code>car.color = white</code>	<code>car.start()</code> <code>car.drive()</code> <code>car.brake()</code> <code>car.stop()</code>



Objects are **variables** too. But objects can contain **many values**.

This code assigns many values (Fiat, 500, white) to a variable name

```
const car = {type:"Fiat", model:"500", color:"white"};
```

The values are written as **name:value** pairs (name and value separated by a colon).



Spaces and line breaks are not important. An object definition can **span multiple lines**:

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 50,  
  eyeColor: "blue"  
};
```



The name:values pairs in JavaScript objects are called properties:

Property	Property Value
firstName	John
lastName	Doe
age	50
eyeColor	blue



You can access object properties in two ways:

```
objectName.propertyName
```

or

```
objectName["propertyName"]
```

Hands-on:

https://www.w3schools.com/js/tryit.asp?filename=tryjs_objects_properties_1



Methods are actions that can be performed on objects.

Methods are stored in properties as function definitions.

Property	Property Value
firstName	John
lastName	Doe
age	50
eyeColor	blue
fullName	function() {return this.firstName + " " + this.lastName;}

```
const person = {  
  firstName: "John",  
  lastName : "Doe",  
  id       : 5566,  
  fullName : function() {  
    return this.firstName + " " + this.lastName;  
  }  
};
```



In JavaScript, the **this** keyword refers to an object.

Which object depends on how **this** is being invoked (used or called).

this is not a variable. It is a **keyword**. You cannot change the value of this.



In a function definition, **this** refers to the "owner" of the function.

In the example above, **this** is the person object that "owns" the fullName function.

In other words, **this.firstName** means the **firstName** property of this object.



You access an object method with the following syntax:

```
objectName.methodName()
```

Example

```
name = person.fullName();
```

Hands-on:

https://www.w3schools.com/js/tryit.asp?filename=tryjs_objects_method



Hãy viết một chương trình JavaScript để quản lý thông tin của một đội bóng. Để làm điều này, bạn cần thực hiện các bước sau:

1. Tạo một đối tượng có tên là **footballTeam** để lưu thông tin về đội bóng. Đối tượng này cần có các thuộc tính sau:

teamName: Tên của đội bóng (string).

coachName: Tên của huấn luyện viên (string).

players: Một mảng chứa thông tin về các cầu thủ của đội bóng. Mỗi cầu thủ là một đối tượng với các thuộc tính:

name: Tên của cầu thủ (string).

age: Tuổi của cầu thủ (number).

position: Vị trí chơi bóng của cầu thủ (string).

isInjured: Một giá trị boolean cho biết liệu cầu thủ có bị chấn thương hay không.



2. Tạo một phương thức (method) có tên là **addPlayer** trong đối tượng **footballTeam** để thêm một cầu thủ mới vào đội bóng. Phương thức này sẽ chấp nhận các thông tin về cầu thủ và thêm cầu thủ đó vào mảng **players**.
3. Tạo một phương thức (method) có tên là **listPlayers** trong đối tượng **footballTeam** để in ra danh sách các cầu thủ trong đội bóng. Phương thức này sẽ duyệt qua mảng **players** và in ra thông tin về mỗi cầu thủ.
4. Tạo một phương thức (method) có tên là **injuredPlayers** trong đối tượng **footballTeam** để đếm và in ra số lượng cầu thủ bị chấn thương trong đội bóng.
5. Tạo một phương thức (method) có tên là **teamInfo** trong đối tượng **footballTeam** để in ra thông tin tổng quan về đội bóng, bao gồm **tên đội bóng, tên huấn luyện viên và số lượng cầu thủ trong đội**.



Choose an idea for your website

Startup company → new kind of business

Popular website → new features

Simple website with only 2-3 pages is not recommended

Survey for similar website

List all the pages they have

What is your impression?

Initialize a git repository